Unidad 3

Mapas (Map)

Colecciones de pares clave-valor con new Map, sus métodos set, get, has y delete, el recorrido con desestructuración y la conversión a array con spread.

Un **mapa** (Map) junta lo mejor de las estructuras anteriores: guarda una colección de pares **clave → valor**, como una agenda donde cada nombre lleva asociado un teléfono, y garantiza que **las claves no se repiten**. Se busca por la clave y se obtiene su valor.

```
const telefonos = new Map();

telefonos.set('ana', '600 111 222');
telefonos.set('luis', '600 333 444');

console.log(telefonos.get('ana'));     // → 600 111 222
console.log(telefonos.size);           // → 2
```

Frente a un array, donde para encontrar el teléfono de Ana habría que recorrerlo comparando nombres, el mapa lo devuelve directamente.

Creación

Se crea con `new Map()` . Para rellenarlo desde el principio se le pasa un **array de arrays**, donde cada array interior es un par `[clave, valor]` :

```
const capitales = new Map([
  ['España', 'Madrid'],
  ['Francia', 'París'],
  ['Italia', 'Roma'],
]);

console.log(capitales.size);           // → 3
```

Las claves pueden ser de cualquier tipo, no sólo cadenas: números, booleanos e incluso arrays u objetos.

Añadir, actualizar y eliminar

`set(clave, valor)` añade un par. Devuelve el propio mapa, así que se puede encadenar. Si la clave **ya existe**, no se duplica: se sobrescribe el valor antiguo con el nuevo:

```
const stock = new Map();

stock.set('tornillos', 100).set('tuercas', 50);
stock.set('tornillos', 80);            // actualiza, no añade

console.log(stock);                    // → Map(2) { 'tornillos' => 80,
                                       //            'tuercas' => 50 }
```

`delete(clave)` elimina el par y devuelve `true` si existía, y `clear()` vacía el mapa:

```
stock.delete('tuercas');               // → true
stock.delete('clavos');                // → false, no estaba
```

Consultar

`has(clave)` dice si la clave está, y `get(clave)` devuelve su valor, o `undefined` si la clave no existe. Las dos son muy rápidas: no recorren nada, van directas a la clave.

```
const capitales = new Map([['España', 'Madrid'], ['Italia', 'Roma']]);

capitales.has('Italia');               // → true
capitales.get('Italia');               // → Roma
capitales.get('Portugal');             // → undefined
```

Como `get` devuelve `undefined` para lo que no está, el patrón habitual para **contar** o **acumular** es preguntar primero con `has`, o partir de 0 cuando `get` no devuelve nada:

```
const votos = new Map();
const papeletas = ['sí', 'no', 'sí', 'sí'];

for (const voto of papeletas) {
  if (votos.has(voto)) {
    votos.set(voto, votos.get(voto) + 1);
  } else {
    votos.set(voto, 1);
  }
}
console.log(votos);                    // → Map(2) { 'sí' => 3, 'no' => 1 }
```

Recorrido

Con `for … of`, cada vuelta entrega el par completo como un array `[clave, valor]`:

```
for (const par of capitales) {
  console.log(par);
}
// → ['España', 'Madrid']
// → ['Italia', 'Roma']
```

Trabajar con `par[0]` y `par[1]` se lee mal. La **desestructuración** permite darles nombre directamente en la cabecera del bucle:

```
for (const [pais, capital] of capitales) {
  console.log(`${capital} es la capital de ${pais}`);
}
// → Madrid es la capital de España
// → Roma es la capital de Italia
```

Si sólo interesan las claves o sólo los valores, los métodos `keys()` y `values()` devuelven un iterable con cada parte:

```
for (const pais of capitales.keys()) {
  console.log(pais);                   // → España, Italia
}

for (const capital of capitales.values()) {
```

```
  console.log(capital);                // → Madrid, Roma
}
```

Necesito…	Uso
Guardar o actualizar	`mapa.set(clave, valor)`
Leer un valor	`mapa.get(clave)`
Saber si existe	`mapa.has(clave)`
Eliminar	`mapa.delete(clave)`
Cuántos pares hay	`mapa.size`
Recorrer pares	`for (const [clave, valor] of mapa)`
Sólo claves o valores	`mapa.keys()` / `mapa.values()`

Conversión con spread

Igual que con los conjuntos, el operador spread convierte un mapa en un array. El resultado es un array de arrays, el mismo formato que acepta el constructor:

```
const pares = [...capitales];
// → [['España', 'Madrid'], ['Italia', 'Roma']]

const copia = new Map(pares);          // mapa nuevo e independiente
```

> ▶ **Mapa u objeto**

Ejemplo 1: contar apariciones con un mapa

Escribe la función contar(lista) que reciba un array y devuelva un Map cuyas claves sean los elementos distintos de lista y cuyos valores sean cuántas veces aparece cada uno. Recorre el array con for … of y, en cada vuelta, usa has o get para saber si la clave ya está y set para actualizar su cuenta.

```
function contar(lista) {
  const cuentas = new Map();

  return cuentas;
}

console.log(contar(['a', 'b', 'a', 'c', 'a']));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 1

Ejemplo 2: recorrer un mapa con desestructuración

Escribe la función masCaro(precios) que reciba un Map con nombres de productos como claves y precios como valores, y devuelva el nombre del producto más caro. Recórrelo con for (const [nombre, precio] of precios). Si el mapa está vacío, devuelve null.

```
function masCaro(precios) {
  let nombre = null;

  return nombre;
}

const carta = new Map([['café', 1.5], ['tarta', 3.8], ['zumo', 2.4]
console.log(masCaro(carta));
```

Ctrl + Enter ejecuta con tests · Ctrl + Shift + Enter sólo el código · Tab indenta · Esc y luego Tab sale del editor

SALIDA DE CONSOLA

```
Sin salida todavía.
```

Los tests se muestran al ejecutar.

▶ Ver solución del ejemplo 2

De este módulo

T3.2 Ejercicios: conjuntos y mapas

Ocho programas con Set y Map, desde contar palabras únicas hasta un sistema de inicio de sesión con bloqueo de cuentas y un analizador de texto con anagramas.
